LANCE : diagnostic détaillé de l'organisation du code

Analyse de la copie de travail par Codex

Dernière révision : 2026-10-03 12:12 (Europe/Brussels, UTC+02:00)

Diagnostic logiciel et plan proposé ; refactoring non implémenté

Résumé

LANCE dispose déjà de frontières utiles entre agent, API, benchmark et entraînement. La difficulté principale est à l'intérieur de ces ensembles : des phases partagent un hôte implicite, le graphe conserve un état global et plusieurs fonctions réunissent orchestration, interprétation des preuves et projection des résultats. L'analyse syntaxique de 175 fichiers Python et la lecture ciblée des propriétaires font apparaître ces concentrations, sans démontrer que le code ancien serait inutilisé. Le premier lot recommandé isole les règles pures de preuve et le contrôle commun des destinations ; le suivant rend explicite le contexte de vérification. Le plan conserve les entrées publiques, les artefacts et les métriques. Les tests locaux et la relecture indépendante étayent le diagnostic ; aucun gain de performance, résultat de laboratoire ou fonctionnement GPU n'est établi.

Table des matières

1 Question et périmètre **1**

2 Méthode et traçabilité **2**

3 Vue d'ensemble : les frontières existent déjà **2**

4 Constats détaillés et corrections d'organisation **3**

5 Architecture cible et ordre de réalisation proposés **7**

6 Limites et décisions encore ouvertes **8**

7 Conclusion **9**

Références **9**

1 Question et périmètre

Une meilleure organisation doit permettre de répondre rapidement à trois questions : où modifier un comportement, quelles dépendances sont nécessaires et quels tests protègent sa signification ? Déplacer une longue fonction dans un autre dossier ne suffit pas si elle continue à lire l'état privé de tout le pipeline.

L'étude couvre `src/agent`, `src/api`, `src/static`, `src/benchmark`, `src/learning`, `model_training`, les scripts Python versionnés ou non ignorés et les tests. Les fichiers de recherche, les

données, les bibliothèques JavaScript tierces et les fichiers ignorés ne sont pas comptés dans les métriques de production. Les conventions de `AGENTS.md`, les index et les guides de phases sont consultés avant la proposition [1].

État étudié. Le commit de base est `86952627599c003b55d122791def90bcb023db7e`. La copie de travail contient des corrections locales non commitées ; le diagnostic les inclut. Ce rapport ne décrit donc pas exactement le seul commit publié. Il ne modifie aucun comportement applicatif et n'exécute ni réseau d'audit, ni laboratoire, ni déploiement, ni entraînement.

2 Méthode et traçabilité

2.1 Mesures reproductibles

Le script `scripts/analysis/code_organisation.py` lit la liste Git des fichiers suivis et non ignorés. Il analyse leur arbre syntaxique Python avec Python 3.12.2, sans importer l'application. Le fichier `structure.json` du présent dossier conserve les chemins, les empreintes SHA-256, les tailles, les fonctions et les imports explicites [2]. Les tests et `scripts/analysis/` sont exclus des tailles de production. Les 161 fichiers Python de tests incluent aussi les fixtures et fichiers d'initialisation.

Une longueur est le nombre de lignes physiques de la définition, incluant commentaires, lignes vides et fonctions imbriquées. Ce n'est ni une complexité cyclomatique, ni un score de qualité. Un nombre d'imports compte les modules internes distincts, pas tous les symboles ni les dépendances transitives.

L'empreinte agrégée des sources Python étudiées est :

`cbdeb9c6c70d4e7a06247587410e29232b2bfce40af4c1cfedde95f8e83c8f9d`.

L'empreinte distincte de `app.js` figure dans le JSON. Le script est réexécuté et son résultat comparé intégralement ; des cas synthétiques vérifient imports relatifs, portées et composantes du graphe.

2.2 Lecture, tests et niveaux de preuve

Les grandes fonctions, les chemins d'appel et les contrats existants sont lus directement. Un relecteur indépendant contrôle les constats et la méthode. Les quatre modules de tests ciblés donnent **79 tests réussis** en 4,62 secondes : routage des phases, destinations, isolation des politiques de connaissance et des artefacts de run [5]. Cette sélection ne remplace pas la suite complète et ne valide pas les refactorings proposés.

Une reproduction locale, uniquement en mémoire, charge deux contextes de découverte successifs, puis remplace `load_lab_context` par un mock. Elle confirme le remplacement du contexte et la demande de chargement implicite après remise à zéro. Le script, ses assertions et sa sortie JSON sont conservés [6]. Aucun appel réseau ou lancement de processus externe n'est effectué. Cette observation ne démontre pas une collision concurrente dans le déploiement courant.

Les mentions **constat** décrivent le code ou une observation locale. Les mentions **proposition** décrivent un changement non implémenté. L'effort et les risques sont appréciés qualitativement ; aucun délai ou gain chiffré n'est estimé sans réalisation.

3 Vue d'ensemble : les frontières existent déjà

La séparation application/entraînement/benchmark est utile et doit être conservée. Les six entrées de phase sont routées explicitement ; les profils *full* et *compact* utilisent ces mêmes entrées. La phase 3 dispose déjà de `AnalysisContext` et de résultats structurés. Son agrégation distingue chargement, normalisation, projection, rendu et écriture. Le rapport partage une entrée bornée entre profils. Les transports du fournisseur, diagnostics et règles de fin possèdent déjà des modules dédiés [3, 4]. Ces évolutions donnent un modèle à prolonger.

Table 1 – Périmètre Python mesuré ; lignes physiques, tests exclus.

Ensemble	Fichiers	Lignes
Agent	107	32 191
Benchmark	24	11 710
API	13	3 076
Apprentissage à partir des runs (`src/learning`)	5	2 402
Entraînement (`model_training`)	9	1 431
Autres modules de `src` et scripts	17	2 823
Total	175	53 633

La figure 1 montre ce que le rangement seul ne résout pas : une phase reste souvent une méthode de l'hôte `Pipeline`. Le contexte explicite de la phase 3 a réduit ce couplage à l'exécution, tandis que son adaptateur capture encore l'état global du graphe.

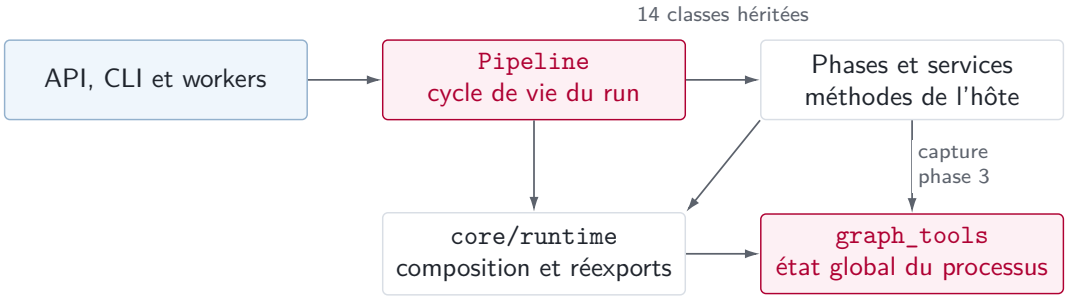


Figure 1 – Architecture actuelle, relations structurelles sélectionnées. Les flèches indiquent dépendance ou lecture d'état, pas l'ordre temporel des six phases. Les réexports de `runtime` et le graphe global sont deux points de couplage ; cette vue ne représente pas tous les imports.

4 Constats détaillés et corrections d'organisation

4.1 Pipeline : des fichiers séparés, un hôte partagé

Constat. `Pipeline` hérite de 14 classes (`src/agent/pipeline.py`, l. 38–53). `registry.run_phase` reçoit encore cet hôte. La phase de vérification lit `run_dir`, appelle des méthodes privées de politique et d'exécution, puis écrit des diagnostics sur `self` (`src/agent/phases/verification/run.py`, l. 36–88). Sa fonction principale couvre 502 lignes. Il faut donc connaître une partie importante du pipeline pour comprendre ou tester une phase isolée.

Conséquence. La frontière du dossier ne garantit pas une frontière d'état. Ajouter une propriété à `self` devient une dépendance disponible implicitement pour toutes les classes héritées. Une extraction mécanique de méthodes conserve cette difficulté.

Proposition. Garder `Pipeline` comme façade publique. Introduire `VerificationContext`, des services bornés et un résultat structuré, sur le modèle existant de la phase 3. Le contexte capture fournisseur, profil, politiques et dossier au début de la phase ; les workers renvoient leurs résultats au coordinateur. Migrer une phase à la fois. Éviter un contexte qui expose seulement tout `Pipeline` sous un nouveau nom.

4.2 Runtime : une composition qui sert aussi de façade métier

Constat. `core/runtime.py`, lignes 7–71, importe déploiement, scénarios, catalogue, fournisseur, graphe, scanner, politiques et règles de preuve. Les phases utilisent ses alias privés pour synthétiser un verdict. Le runner importe aussi `COMPACT_INTRUSION_COMPLETION_TOOL` depuis une implémentation

concrète de phase (`src/agent/core/runner.py`, l. 9). La direction des dépendances est donc plus mélangée que la description « infrastructure » ne le suggère.

Conséquence. Un helper pur n'a pas besoin de la composition du moteur, mais sa consommation via `runtime` brouille son propriétaire. Un outil de fin spécifique apparaît comme une connaissance du runner commun.

Proposition. Importer les fonctions pures directement depuis leur propriétaire. Réserver la composition du catalogue et les adaptateurs de cycle de vie à un endroit explicite. Transmettre la règle de fin dans un plan de phase, ou la placer dans un petit contrat partagé. Conserver temporairement les alias appelés par les adaptateurs et tests, puis les supprimer après migration des usages. Le catalogue pur commun de `src/benchmark/tool_registry.py` est intentionnel : un import agent vers ce module ne constitue pas à lui seul une fuite de vérité terrain.

4.3 Graphe : état global et initialisation implicite

Constat. `graph_tools.py`, lignes 31–64, conserve neuf champs globaux de contexte, dont topologie, découverte, résultats CVE et graphe pondéré. Chaque chargement remet ces champs à zéro. La capture d'agrégation lit directement `graph_tools._scenario_topology`. Une lecture non préparée peut appeler `_ensure_loaded`, puis `load_lab_context` (lignes 84–96 et 351–353). La reproduction en mémoire confirme ces deux mécanismes [6].

Conséquence. Deux objets de run dans le même processus ne possèdent pas intrinsèquement deux contextes de graphe. La règle actuelle d'un seul run actif dans l'API limite un usage concurrent courant ; elle ne transforme pas les globals en données appartenant au run. L'initialisation cachée complique aussi les tests et la lecture des effets de bord.

Proposition. Créer un `GraphContext` par run, puis lier les outils à cet objet. Conserver le snapshot public minimal déjà utilisé par l'agrégation. Les opérations de lecture reçoivent un contexte préparé ; les chargements de laboratoire deviennent explicites. Ne pas exposer la vérité terrain au worker pour simplifier cette migration. Tester deux contextes distincts en mémoire avant d'envisager toute évolution de concurrence réelle.

4.4 Preuves et destination : rendre l'autorité sémantique explicite

Constat. `evidence/capabilities.py`, ligne 43, importe dans une fonction le helper privé `_credential_result_supported` du grand module `exploit_evidence.py`. Celui-ci importe en retour `capabilities` au niveau module. Les règles d'authentification partagées entre vérification, accès observés et évaluation dépendent ainsi d'un synthétiseur plus général. Par ailleurs, `core/executor.py`, ligne 14, consomme une règle commune de destination située dans `phases/intrusion/scope.py`. Son périmètre est désormais plus large que cette phase et plusieurs messages parlent encore de « Phase 5 ».

Conséquence. Un développeur peut chercher ou recopier une même règle dans plusieurs endroits. Le chemin et les noms rendent son périmètre ambigu. Les corrections locales récentes de liaison entre service et preuve augmentent la nécessité de garder une seule autorité, sans changer ses critères pendant la réorganisation.

Proposition. Déplacer les critères d'authentification vers `evidence/credentials.py` ou un module pur `evidence/authentication.py`. Faire dépendre les synthèses et capacités de cette autorité commune. Déplacer le contrôle commun des destinations vers `core/scope.py`, conserver une compatibilité transitoire et harmoniser ensuite noms et documentation. Cette extraction est le premier lot recommandé : ciblée et protégée par des tests de verdict déjà disponibles.

Limite du graphe d'imports. Aucune composante circulaire n'apparaît avec les seuls imports explicites de portée module. Trois apparaissent en ajoutant la portée fonction : `capabilities` et `exploit_evidence` ; `recon_tools` et `tool_loader` ; `scenario_deployment` et `scenario_exports`. La portée est lexicale : `_load_recon_tools` contient un import, mais est appelée pendant le

chargement du module. Ce relevé ne prouve donc pas que tous ces imports sont exécutés tardivement. Il ne modélise pas les packages parents implicites ni `importlib` ou `__import__`. Aucun échec d'import circulaire n'est observé lors des vérifications ciblées.

4.5 Grandes fonctions : séparer décisions, état et projection

Le tableau 2 localise les concentrations les plus utiles à traiter. Vingt-quatre fonctions non imbriquées dépassent 200 lignes dans le périmètre Python. Leur longueur guide la lecture ; c'est le mélange de responsabilités constaté qui motive une extraction.

Table 2 – Définitions longues sélectionnées, mesurées dans la copie de travail.

Module sous `src/`	Fonction, ligne de début	Lignes
`benchmark/evaluator.py`	`evaluate`, 1954	858
`agent/provider.py`	`_openai_loop`, 408	792
`agent/exploit_evidence.py`	`synthesize_exploit_result`, 959	720
`agent/phases/intrusion/compact.py`	`_apply_compact_intrusion_tool_contract`, 130	690
`agent/phases/verification/run.py`	`_run_exploit_agents`, 36	502
`agent/pipeline.py`	`_execute_run`, 462	364

Constats. La synthèse de preuves regroupe liaison à la cible, interprétation de plusieurs protocoles et verdict final. Le scanner de 2 297 lignes regroupe exécution et de nombreux extracteurs protocolaires déjà nommés séparément. Le contrat d'intrusion compacte encapsule état de campagne, vérifications, outils enveloppés et décision de fin dans une seule méthode. La boucle du fournisseur possède 26 paramètres, `self` inclus, et reconnaît directement des marqueurs de reconnaissance, vérification et intrusion (`src/agent/provider.py`, l. 1092–1151). Le transport et les règles de complétion sont déjà partiellement extraits ; le reste est principalement leur coordination avec des contrats de phase.

Propositions. Dans les preuves, séparer les évaluateurs par famille de protocole, avec une liaison cible/service commune et un résultat typé ; garder `synthesize_exploit_result` comme entrée stable. Dans le scanner, extraire les parseurs existants par protocole avant de toucher l'ordonnancement. Dans l'intrusion, représenter explicitement `CampaignState` et les transitions, puis produire les projections depuis cet état. Dans le fournisseur, introduire un plan de boucle et une politique de complétion cohérente avec `core/completion_policy.py`. Une classe qui encapsulerait les mêmes 26 indicateurs sans clarifier leurs combinaisons n'apporterait qu'un gain de signature. Les tests doivent préserver annulation, budgets, sauvegarde refusée et reprise, pas seulement le succès nominal.

4.6 Benchmark : conserver une évaluation publique, rendre ses étapes pures

Constat. `evaluator.py` couvre 3 066 lignes. Sa fonction `evaluate` lit vérité terrain et contrats, contrôle leur identité, charge les observations, associe les preuves aux affirmations, calcule correspondances et métriques, puis construit le résultat. L'API gère elle-même une empreinte de dépendances et le cache du score (`src/api/routes/runs.py`, l. 374–470). `src/learning/error_mining.py` importe également le chargeur privé `_load_llm_findings` de l'évaluateur.

Conséquence. Les consommateurs dépendent d'un gros module et parfois de son détail de chargement. Une extraction de règles vers un nouveau chemin peut aussi rendre le cache incomplet si sa liste de sources n'évolue pas.

Proposition. Conserver `evaluate` et le type public du résultat. Extraire d'abord les entrées dans un objet `EvaluationInputs`. Séparer ensuite admission des preuves, correspondances et calculs de métriques. Les étapes pures reçoivent des données déjà chargées ; une API de lecture des observations remplace les imports privés de l'apprentissage. Le service d'évaluation possède sa version ou sa liste

de dépendances pour l'empreinte du score. L'API conserve la projection spécifique au tableau et son adaptation HTTP. Ne jamais faire remonter la vérité terrain de l'évaluateur vers le contexte agent.

4.7 API : une route héberge un coordinateur d'application

Constat. `routes/pipeline.py` couvre 811 lignes et possède `_state`, verrou, threads, démarrage, batch, nettoyage et publication des événements SSE (*Server-Sent Events*, flux serveur vers navigateur). `activity.py`, lignes 30–32, et `scenario_generator.py`, lignes 155–162, lisent cet état privé. `routes/runs.py`, 967 lignes, combine lecture des artefacts, sécurité des chemins, scénarios, cache, scores, évaluation LLM et téléchargement ZIP.

Conséquence. Un appel HTTP, une transition de run et une notification du thread sont difficiles à comprendre séparément. Une autre route connaît l'état interne de sa voisine au lieu de dépendre d'un service explicite. Cela n'établit pas, à lui seul, une course ou un défaut de verrouillage.

Proposition. Extraire un `RunCoordinator` et un stockage d'état applicatif avec `snapshot`, `start`, `stop` et finalisation. Conserver la règle actuelle d'un seul run actif. Isoler le pont thread/boucle asynchrone/événements, puis un service de lecture des runs ; garder les protections de chemins et symlinks à une frontière testée. Les routes valident les requêtes et traduisent réponses et erreurs. La séparation n'introduit pas un nouveau modèle de déploiement ni plusieurs runs simultanés.

4.8 Interface : découper le JavaScript selon les fonctionnalités

Constat. `src/static/app.js` contient 3 761 lignes physiques. Il gère état global, graphe Cytoscape, démarrage et suivi des runs, détails, benchmark, modèles, fournisseurs, session d'administration et persistance locale. Les helpers réseau `fetchJSON`, `adminFetch` et `apiSend` sont groupés vers les lignes 3315–3367. L'entrée HTML charge un script classique ; des fonctions génèrent aussi leur propre balisage et leurs écouteurs. `activity.js` et `scenario_generator.js` sont déjà séparés.

Conséquence. Ajouter une fonctionnalité risque de mêler rendu, stockage et appels réseau dans un même fichier. Il n'existe pas une frontière d'import claire entre état de run, graphe et benchmark.

Proposition. Garder le JavaScript natif. Extraire un client API commun, puis les fonctionnalités `run`, `topology`, `benchmark` et `models`, avec un état explicite. `app.js` devient le point de composition et de branchement des événements. Migrer progressivement vers les modules ES en vérifiant le chargement, les références entre scripts et les éventuels handlers dans le HTML généré. Le découpage n'impose pas une migration React ou un bundler ; leurs coûts ne sont pas justifiés par ce diagnostic. Vérifier visuellement les écrans et leur reconnexion après rechargement lors de la future implémentation.

4.9 Ancien code, entraînement, tests et documentation

Constats. Les programmes d'entraînement sont séparés de l'application. Leur README dit explicitement que le parcours QLoRA antérieur et les scripts de préparation, mutation, inspection et inférence sont conservés. Parmi eux, `analyze_adapter.py` charge modèle et adaptateur au niveau module ; `fix_jsonl.py` lit un chemin local fixé au niveau module. Ils doivent être inspectés statiquement avant tout import ou exécution. Aucun entraînement GPU n'est testé ici.

L'absence d'import entrant n'établit pas qu'un module est mort : `src/agent/moe_server.py` n'en possède aucun dans le graphe mesuré, mais le guide HMoE le lance directement. Les CLI et handlers configurés constituent d'autres usages possibles [7]. Aucune suppression n'est donc recommandée sans inventaire complet des commandes, manifestes, configurations, tests et procédures opérationnelles.

Les tests offrent déjà des protections de contrat utiles. Certains vérifient aussi le chemin `__module__` ou l'absence d'un ancien fichier (`tests/test_phase_modules.py`). Ces assertions protègent une décision d'architecture ; elles devront changer lors d'un déplacement autorisé, sans être confondues avec une régression de comportement.

Propositions. Classer les outils d'entraînement par rôle et statut, avec entrées `main`, arguments explicites et aucun chargement modèle à l'import pour les utilitaires réutilisables. Ne pas déplacer le serveur MoE ni les chemins GPU avant vérification des commandes externes. Organiser progressivement les tests par propriétaire (preuves, moteur, API, benchmark), en gardant des tests transversaux pour les frontières. Une page par module doit indiquer responsabilité, entrée, artefacts et invariants ; le diagnostic et la proposition restent dans `research`. Corriger le vocabulaire « intrusion compacte » du contrôle commun de destination lorsque ce lot est réalisé. Éviter de documenter une architecture proposée comme déjà acquise.

5 Architecture cible et ordre de réalisation proposés

5.1 Frontières, sans réécriture générale

La figure 2 propose une direction des dépendances. Les phases reçoivent des entrées et services explicites ; elles utilisent les règles pures partagées. L'évaluateur utilise ces mêmes règles, mais conserve seul sa vérité terrain et ses contrats de correspondance. Le contexte de graphe appartient à la session de run. Les noms illustrent des responsabilités, pas une arborescence définitivement retenue.

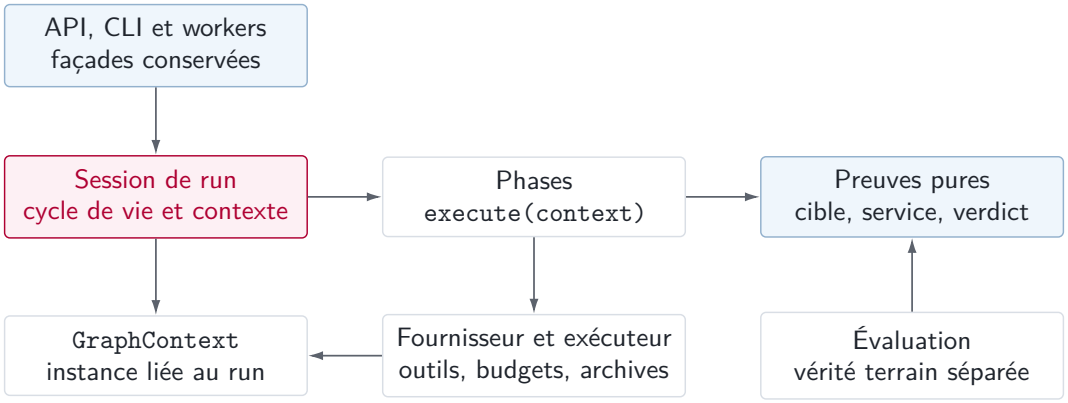


Figure 2 – Architecture proposée, non implémentée. Les flèches indiquent les dépendances à rendre explicites. L'évaluation partage les critères de preuve, sans transmettre son oracle aux phases. Le dessin omet volontairement les adaptateurs de compatibilité et les détails internes de chaque phase.

Une arborescence possible ajoute `core/scope.py`, `tools/graph_context.py` et `phases/verification/context.py`. Elle complète `evidence/` et les modules de complétion existants plutôt que créer un dossier `utils` commun. Côté API, `services/` porte coordination et lecture ; côté benchmark, un sous-ensemble `evaluation/` porte les étapes pures tandis que `evaluator.py` conserve l'entrée. Ces chemins seront confirmés par le premier lot, après contrôle des imports, du cache de score et des contrats de tests.

5.2 Lots et critères d'acceptation

Les lots suivants sont ordonnés selon leurs dépendances. L'interface et l'inventaire du code ancien peuvent avancer indépendamment ; leur réalisation n'exige pas de lancer un laboratoire.

1. **Autorité des preuves et portée commune.** Extraire les critères d'authentification et déplacer le contrôle commun des destinations. Critères : verdicts identiques sur preuves positives, refus, résultats incomplets et mauvais service ; mêmes restrictions full/compact ; aucune dépendance de l'autorité pure vers le synthétiseur général. Risque modéré, changement ciblé ; préserver les alias nécessaires pendant migration.
2. **Une phase réellement isolable.** Introduire le contexte et le résultat de vérification, sans changer son planning ni ses outils. Critères : exécution testable sans construire `Pipeline`, dossiers

et politiques capturés une fois, erreurs de workers agrégées explicitement, statuts et artefacts préservés pour les deux profils. Ce lot confirme le modèle avant de le généraliser.

3. **État du graphe appartenant au run.** Créer l'instance de contexte et les outils liés, puis supprimer les lectures privées globales. Critères : deux contextes mémoire indépendants, aucune lecture entraînant un chargement implicite, snapshot d'agrégation identique, topologie publique préservée. Le verrou de laboratoire et le mode single-run de l'API restent en place.
4. **Contrat compact et boucle fournisseur.** Extraire état et transitions d'intrusion, puis transmettre une politique de complétion au fournisseur. Critères : même liste d'outils autorisés à chaque transition, mêmes cas de reprise après fin refusée, annulation et budgets respectés, mêmes preuves archivées. Éviter de modifier simultanément les critères de réussite d'une action.
5. **Évaluation et services API.** Extraire les entrées et étapes de l'évaluation avant de sortir cache et coordination des routes. Ces sous-lots peuvent être revus séparément. Critères : mêmes résultats strict-v3 sur corpus local figé, empreinte de cache invalide après changement de chaque source sémantique, mêmes codes HTTP, sécurité des chemins, replay SSE et fin de run. La lecture des anciens artefacts reste compatible.
6. **Modules de l'interface.** Extraire client API et fonctionnalités, avec `app.js` comme composition. Critères : démarrage, arrêt, reconnexion, graphe, historique, tableau benchmark et gestion des modèles fonctionnent ; inspection navigateur aux tailles utilisées et tests de session d'administration. Les fonctionnalités réseau réelles se vérifient avec mocks ou fixtures locales avant toute campagne.
7. **Nettoyage et navigation.** Établir un inventaire conservé/actif/à confirmer, puis déplacer tests et documentation avec leurs liens. Critères : chaque suppression possède une preuve d'absence d'usage dans commandes/configurations et une décision documentée ; aucun wrapper de compatibilité n'est supprimé par sa seule ancienneté. Les parcours GPU conservés restent repérables.

5.3 Validation à associer aux futurs lots

Les tests ne doivent pas simplement recopier l'organisation retenue. Ils protègent les entrées, sorties et invariants à la frontière déplacée. Pour les preuves, les tests d'authentification, de liaison service/preuve, d'accès et de destination sont prioritaires. Pour les phases, préserver statuts, annulation, budgets, artefacts et distinction entre observations et affirmations. Pour l'évaluateur, comparer le résultat complet sur des fixtures figées, pas seulement le score final. Pour les routes, couvrir isolation, symlinks, invalidation du cache et replay d'événements. Pour le frontend, compléter les tests par une inspection navigateur.

La suite prescrite par le dépôt est `python -m pytest -q tests model_training/tests`, accompagnée de `git diff --check`. Elle devra être exécutée après les futurs changements selon leur périmètre. Les 79 tests de cette étude décrivent seulement l'état analysé ; aucune compatibilité d'une future implémentation n'est annoncée.

6 Limites et décisions encore ouvertes

La revue est approfondie sur les frontières et modules concentrés, mais n'est pas une lecture exhaustive de chaque branche de 53 633 lignes. Les scripts et commandes externes, configurations distantes et utilisateurs historiques peuvent fournir des usages invisibles au graphe. Le relevé ne prouve pas une absence de code mort ; il ne permet pas de supprimer du code sur la base d'un degré entrant nul.

Le probe de graphe est synthétique. Son audit hook vérifie les connexions, processus et ouvertures de fichiers en écriture qu'il observe ; ce n'est pas une sandbox native exhaustive. Le chargement réel du laboratoire est remplacé par un mock. La stabilité en production, les débits du fournisseur, les effets sur les durées et l'entraînement GPU ne sont pas mesurés. Les métriques de taille doivent rester reproductibles, sans devenir des objectifs de réduction arbitraire.

L'architecture cible et l'ordre proposé sont des décisions à éprouver par petits lots. Le premier lot doit fixer la frontière de preuve ; le deuxième doit établir si un contexte de vérification suffit ou

révèle des services supplémentaires. La suppression d'aliases, la granularité des modules frontend et le regroupement des utilitaires d'entraînement restent ouverts.

7 Conclusion

L'organisation du dépôt est compréhensible au niveau des grands ensembles. Le besoin principal est de rendre les responsabilités et l'état explicites à l'intérieur du moteur, puis dans l'API et l'interface. La phase 3 fournit déjà un précédent utile. La suite recommandée commence par les règles pures partagées, poursuit par une phase isolable et un graphe appartenant au run, puis traite les contrats compacts, l'évaluation et les consommateurs.

Les suppressions de code ancien doivent suivre une vérification d'usage, et les tests de contrat doivent accompagner chaque lot. Cette stratégie permet des changements relisibles sans mélanger rangement, nouvelles règles de preuve et nouveaux résultats expérimentaux.

Références

[1] Dépôt LANCE. `AGENTS.md`, `README.md`, `docs/README.md`, `research/README.md`. Copie de travail consultée le 3 octobre 2026 ; règles de rangement et périmètres, pas résultats expérimentaux.

[2] `scripts/analysis/code_organisation.py` et `research/2026-10-03-organisation-code/structure.json`. Mesures AST reproduites le 3 octobre 2026, Python 3.12.2 ; empreintes de chaque fichier et commit de base conservés dans le JSON.

[3] Dépôt LANCE. `src/agent/pipeline.py`, `src/agent/phases/registry.py`, `src/agent/phases/README.md`, `src/agent/phases/analysis/context.py`, modules d'agrégation et `src/agent/phases/verification/run.py`. Version de travail empreintée par la mesure, incluant les corrections locales.

[4] Dépôt LANCE. `src/agent/core/runtime.py`, `src/agent/core/runner.py`, `src/agent/provider.py` et `src/agent/core/completion_policy.py`. Même version de travail ; lecture de la coordination et des contrats, sans appel à un modèle externe.

[5] Dépôt LANCE. `tests/test_phase_modules.py`, `tests/test_scope_destination_boundary.py`, `tests/test_knowledge_policy_isolation.py` et `tests/test_run_artifact_isolation.py`. Exécution locale du 3 octobre 2026 : 79 tests réussis ; synthèse et commande conservées dans le README.

[6] `scripts/analysis/graph_context_probe.py`, `research/2026-10-03-organisation-code/graph-context-probe.json` et `src/agent/tools/graph_tools.py`. Reproduction en mémoire du 3 octobre 2026 ; chargement du laboratoire remplacé, aucune campagne exécutée.

[7] Dépôt LANCE. `model_training/README.md`, `model_training/analyze_adapter.py`, `model_training/fix_jsonl.py`, `docs/training/hmoe-openwebui.md`. Lecture statique des parcours conservés et commandes documentées ; aucun modèle chargé sur GPU.